UNIVERSIDAD NACIONAL DE MORENO
Departamento de ciencias Básicas y Tecnología

Carrera de Ingeniería en Electrónica

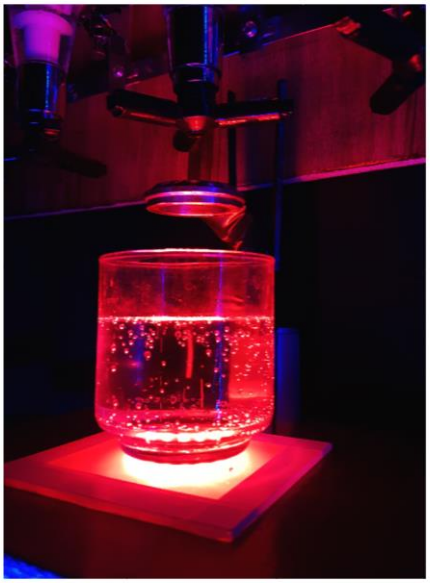
ASIGNATURA: Técnicas digitales II

Año 3º Curso 2017

Docente: Ing. Cristian Conejeros

PROYECTO EN AVR

Máquina de hacer tragos



INTEGRANTES:

Luciano Gutiérrez

Natalia Gonza

Máquina de hacer tragos

El proyecto consta de la construcción y programación en AVR de una máquina que permita realizar diferentes tipos de tragos de manera automatizada, pudiendo el usuario elegir qué tipo de trago desea que la maquina le prepare.

A continuación se encabezará el listado de materiales a utilizar en el proyecto y una breve descripción de su funcionalidad.

Presentación de materiales a usar:

- **Arduino Mega 2560:** La placa arduino configurada para trabajar en AVR con un timer definido en 8Mhz. La base de la programación estará dada mediante el programa Eclipse. Dicho Microcontrolador será el cerebro del proyecto. Usando el ATmega2560 en un nivel más complejo y habiendo analizado las distintas entradas y salidas del micro, será el encargado de realizar y comandar los diferentes periféricos conectados a él.



- **Bluetooth HC05:** se pretende usar este módulo para que el usuario realice la conexión correspondiente y pueda determinar qué tipo de trago se desea preparar. Dicho dispositivo estará conectado en los puertos del micro correspondiente a la UART: TX y RX.



- **Motor paso a paso nema 17 (bipolar):** El motor, junto con un sistema de correa y base, funcionaría como soporte para realizar el desplazamiento del vaso en el que se depositaran las diferentes bebidas.



- **Módulo A4988:** Su función es poder facilitar el control del motor paso a paso.



- **Tira de LED NEOPIXEL**: Se utilizará en la base donde se depositará el vaso, con el fin de que acompañe el proceso de elaboración y finalización del trago.
  Se conectará al micro en los puertos destinados a la comunicación SPI, permitiendo tener un control de cada led de manera autónoma y usando pocos periféricos.



- **Varillas:** se utilizarían para la construcción de las guías que permitirán el desplazamiento del soporte que contendrá el vaso. Dicha base construida irá acompañada de un juego de ruedas para soportar el peso de la base.




- **Dispensador de bebidas:** Dispositivo que nos permite contener las botellas de bebidas, que junto con un sistema de válvulas nos permitirá medir la cantidad de líquido a verter en el vaso.



- **Servo:** Es el encargado de abrir la puerta de la máquina de hacer tragos, para colocar el vaso y retirarlo una vez finalizado el trago.



- **Correa:** junto con el motor paso a paso y una base, formarían parte del sistema de desplazamiento del vaso.



- **Tira de led RGB:** Destinado a realizar la iluminación del dispositivo en general



- **Actuador lineal 50mm:** Usado para activar las válvulas del dispensador de bebidas.



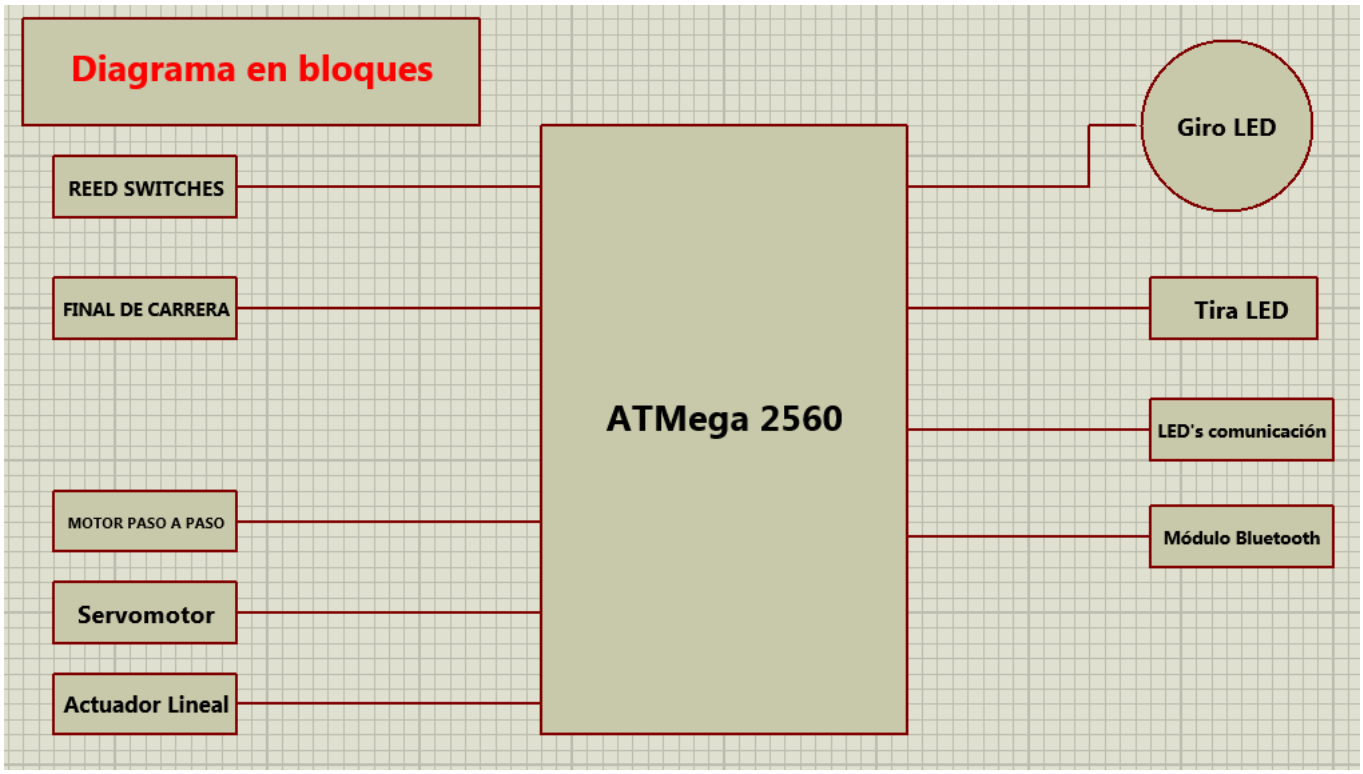
- **Reed Switch y final de carrera:** Están destinados al sensado de la posición del carro que desplaza el vaso, para que se detenga exactamente en cada válvula, evitando contar pasos del motor.





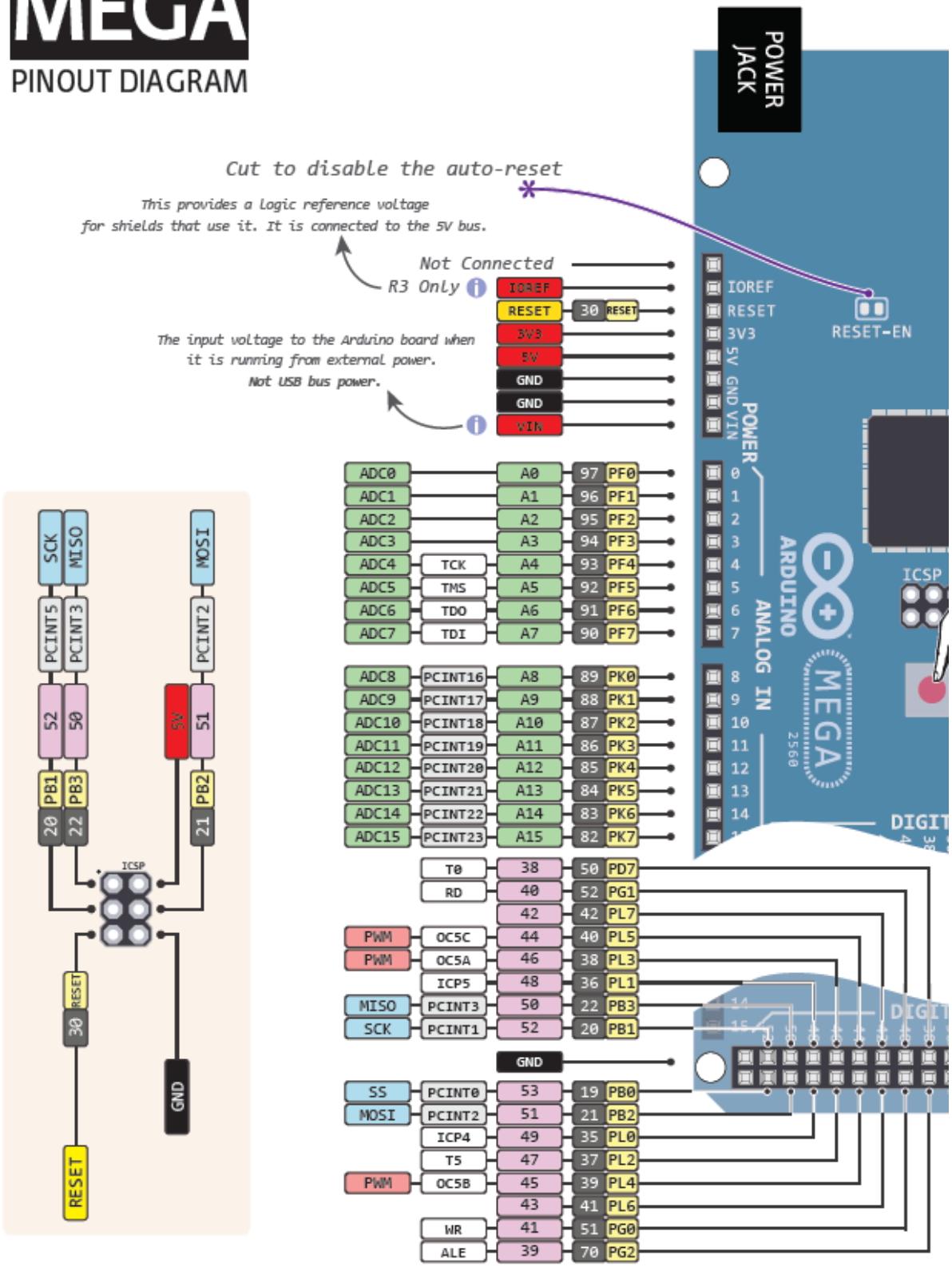
Configuración del micro y puertos a usar

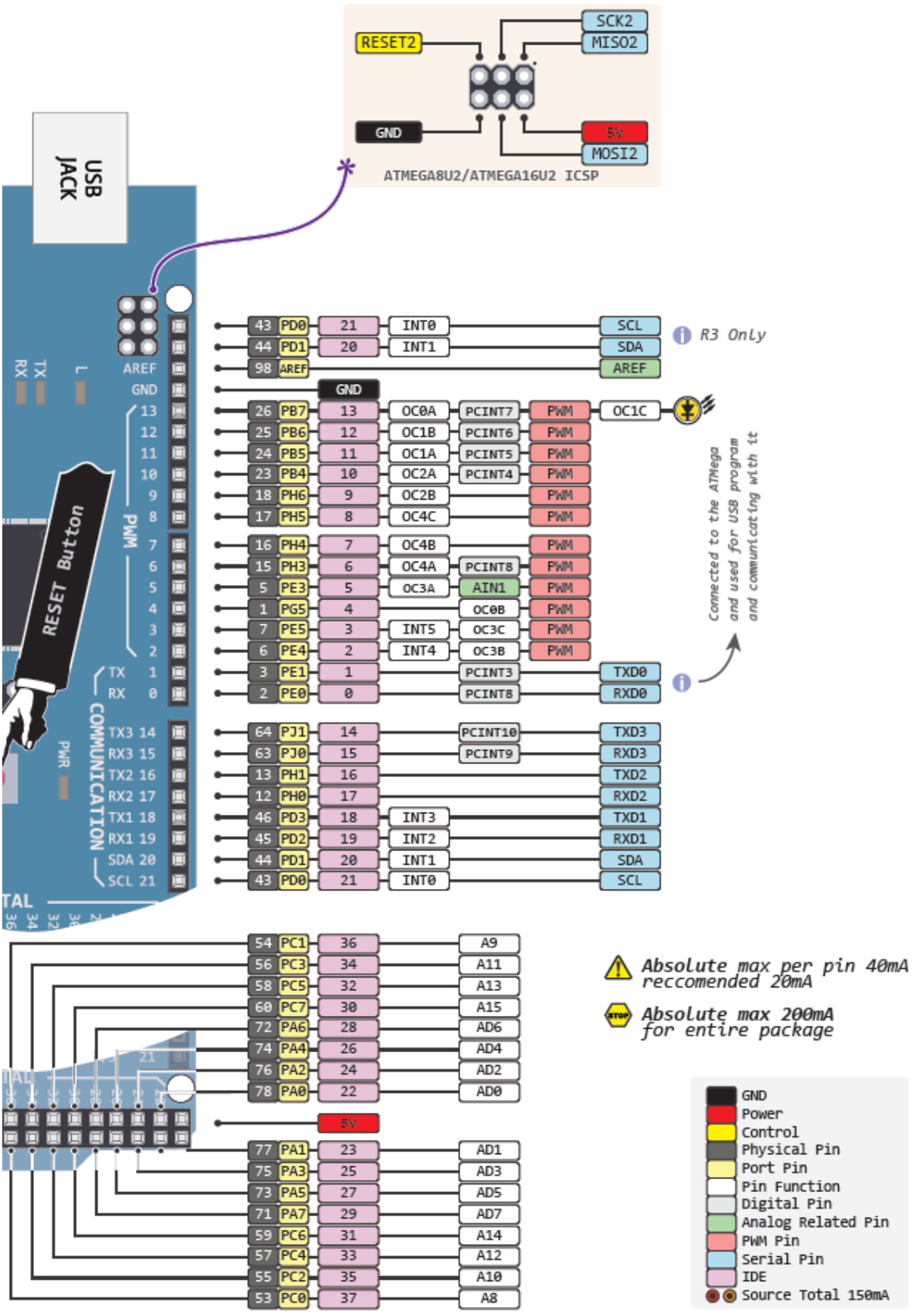
Puertos disponibles del micro ATMega2650 según la configuración de la placa arduino mega.
Contemplando los puertos disponibles se realizó la asignación de los mismo teniendo en cuenta las funciones que cumplirían cada dispositivo a utilizar.



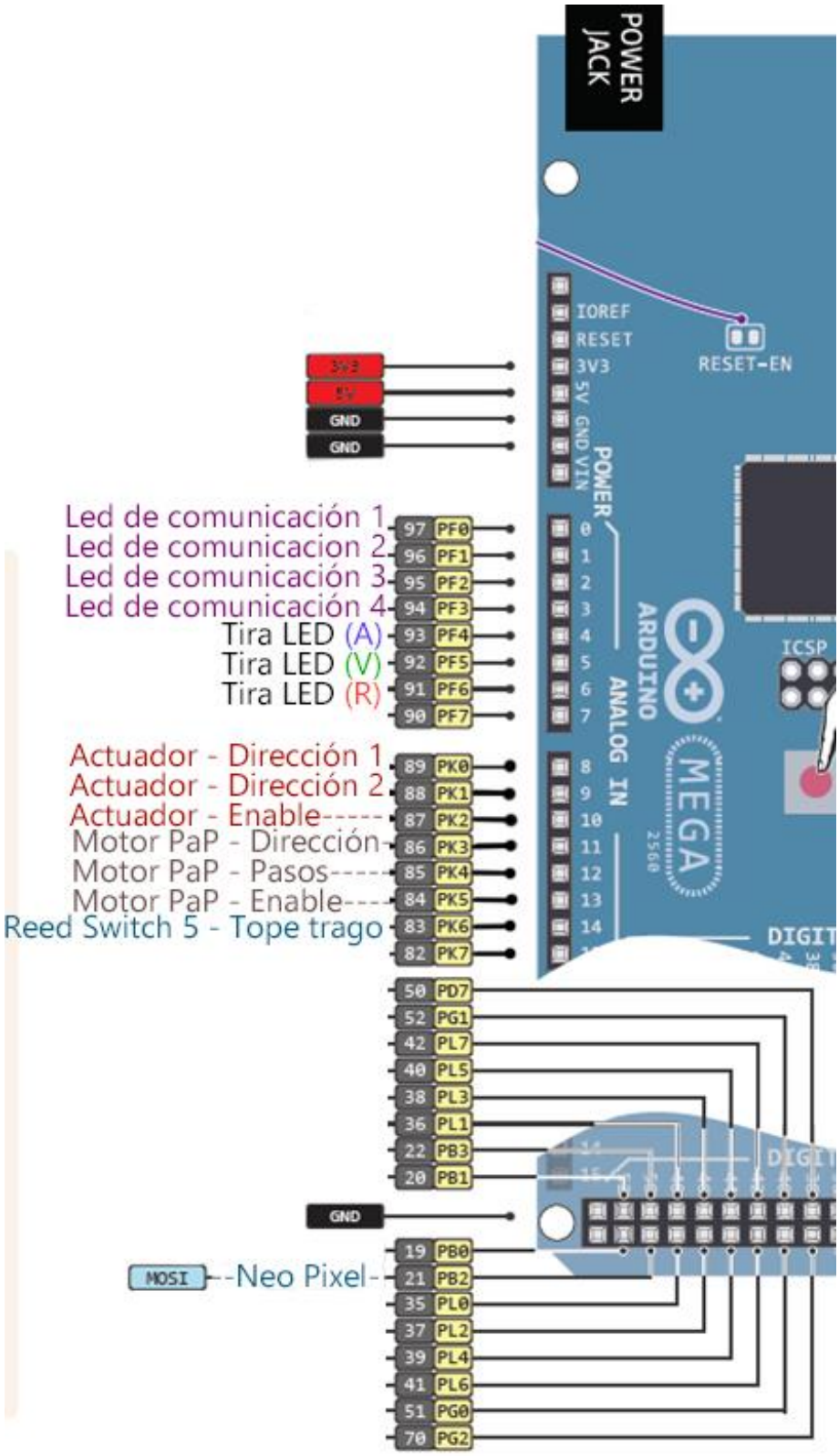
ARDUINO MEGA

PINOUT DIAGRAM




RESET2
SCK2
MISO2
GND
5V
MOSI2
ATMEGA8U2/ATMEGA16U2 ICSP
USB JACK
RESET Button
PWR
COMMUNICATION
PWM
AREF
GND
SCL
SDA
R3 Only
Connected to the ATMega and used for USB program and communicating with it
Absolute max per pin 40mA reccomended 20mA
Absolute max 200mA for entire package
GND
Power
Control
Physical Pin
Port Pin
Pin Function
Digital Pin
Analog Related Pin
PWM Pin
Serial Pin
IDE
Source Total 150mA

Puertos definidos



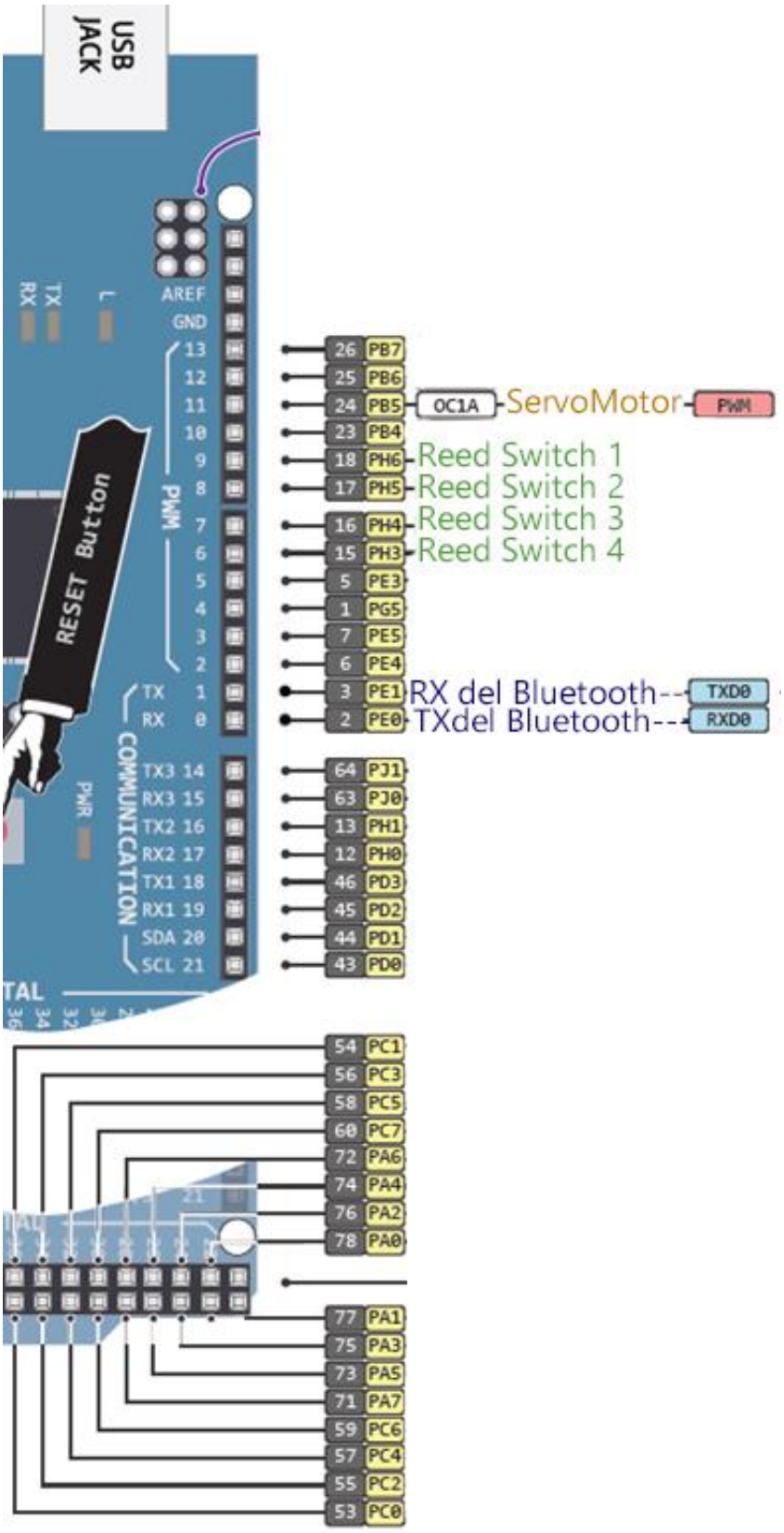
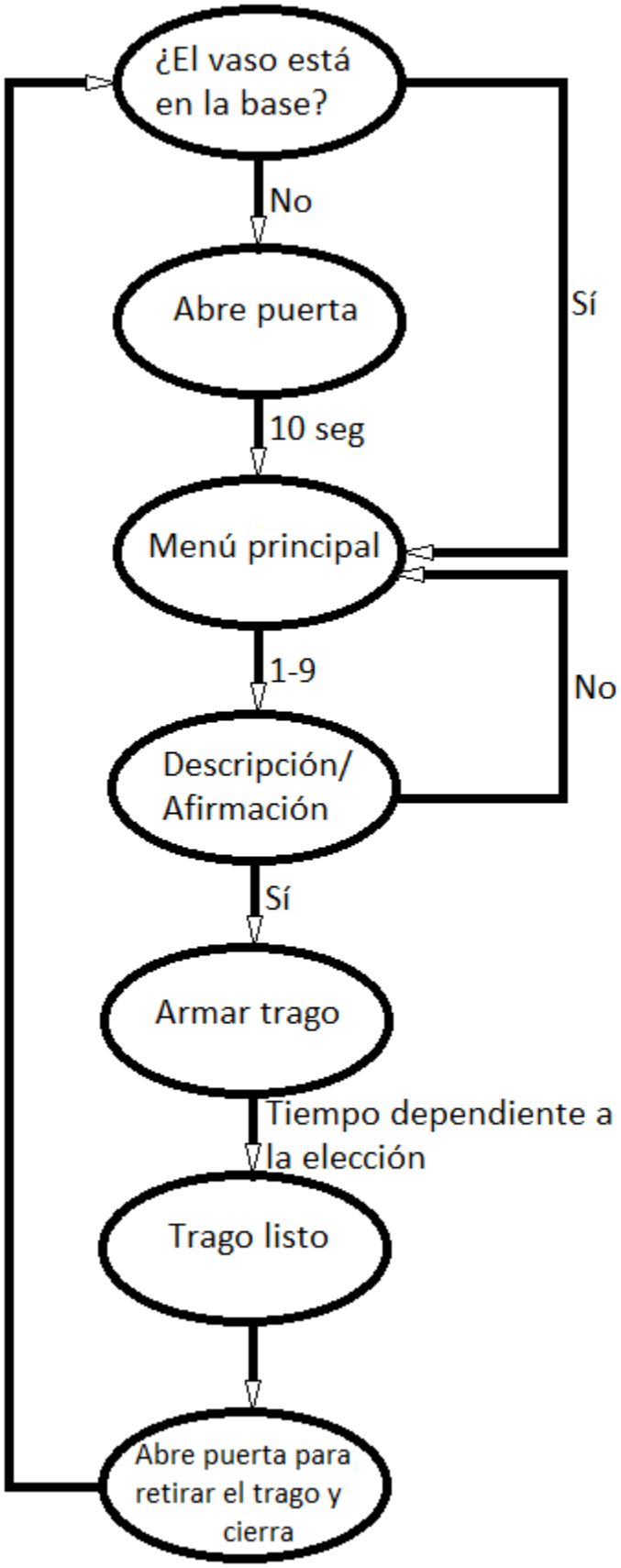

USB JACK
RESET Button
AREF
GND
PWM
COMMUNICATION
TX
RX
PWR
26 PB7
25 PB6
24 PB5
OC1A
ServoMotor
PWM
23 PB4
18 PH6
Reed Switch 1
17 PH5
Reed Switch 2
16 PH4
Reed Switch 3
15 PH3
Reed Switch 4
5 PE3
1 PG5
7 PE5
6 PE4
3 PE1
RX del Bluetooth--
TXD0
2 PE0
TXdel Bluetooth--
RXD0
64 PJ1
63 PJ0
13 PH1
12 PH0
46 PD3
45 PD2
44 PD1
43 PD0
54 PC1
56 PC3
58 PC5
60 PC7
72 PA6
74 PA4
76 PA2
78 PA0
77 PA1
75 PA3
73 PA5
71 PA7
59 PC6
57 PC4
55 PC2
53 PC0

Diagrama de flujo



Descripción de los dispositivos conectados – código:

- ***Servomotor*** : Para controlar el servo es necesario generar una onda cuadrada de frecuencia de 50Hz. En ese caso usamos el *Timer1* -de 16bits- que nos permite manejar con mayor precisión el servo. Lo configuramos en *Modo Fast PWM*, *modo invertido*, preescaler de 8 y con una cuenta terminal (*ICR1*) de 19999.

```
TCCR1A|=(1<<WGM11)|(1<<COM1A1)|(1<<COM1A0);
TCCR1B|=(1<<WGM13)|(1<<WGM12)|(1<<CS11);
ICR1=19999;
```

  El puerto usado corresponde al *Timer1* que la placa presentaba: *PORTB 5*

  Es importante saber que para generar los grados que se va a mover el servo, se debe generar pulsos con un ancho de pulso específico. El ancho del pulso está dado por el valor de ICR1, por lo que modificando este valor logramos obtener el grado de apertura o de cierre del servo.

```
#define Servo_0 OCR1A=ICR1-2899  // Si OCR1A = 17000 es 1ms
#define Servo_90 OCR1A=ICR1-1799 // Si OCR1A = 18000 es 1.5ms
#define Servo_180 OCR1A=ICR1-859 // Si OCR1A = 19000 es 2ms
```

  El servo lo usamos para abrir y cerrar la puerta de la máquina de hacer tragos. Permitiendo colocar el vaso para realizar la bebida y sacarlo una vez finalizado el proceso.

- ***Tira NEOPixel:*** Para la utilización de la tira de led NeoPixel ws2812, fue importante usar librerías, llamadas*: Light_ws2812.c ; Light_ws2812.h y ws2812_config.h*

  Lo que incluía la configuración de comunicación SPI, para poder enviar la información de encendido y apagado de los LED RGB de manera sincronizada.

  La conexión al puerto MOSI de la placa se encuentra en el *PORTB 2*.

  Para la programación usamos funciones creadas por nosotros que nos permiten tener un mayor control del tiempo de encendido, tiempo de apagado y color del mismo y cantidad de vueltas.

  Ejemplo:

  Prototipo:

```
void GiroLedRojo(int CantidadGiros,int Sentido,int RapidezIda,int VueltaEnable,int RapidezVuelta);
```

Función:

```
void GiroLedRojo(int CantidadGiros,int Sentido,int RapidezIda,int VueltaEnable,int RapidezVuelta){
        for(int k=1;k<=CantidadGiros;k++){

        if(Sentido)
        {
        for(int i=0;i<=15;i++)
                  for(int j=20;j<125;j=j+RapidezIda)
                 {
                  led[i].r=j;led[i].g=0;led[i].b=0;
                 ws2812_setleds(led,16);
                 _delay_us(50);
                 }
                if(VueltaEnable)
                {
                  for(int i=0;i<=15;i++)
                          for(int j=125;j>=2;j=j-RapidezVuelta)
                         {
                          led[i].r=j;led[i].g=0;led[i].b=0;
                         ws2812_setleds(led,16);
                         _delay_us(50);
                         }
                }
        }
        else
        {
        for(int i=15;i>=0;i--)
                  for(int j=20;j<125;j=j+RapidezIda)
                 {
                  led[i].r=j;led[i].g=0;led[i].b=0;
                 ws2812_setleds(led,16);
                 _delay_us(50);
                 }
                if(VueltaEnable)
                {
                  for(int i=15;i>=0;i--)
                          for(int j=125;j>=2;j=j-RapidezVuelta)
                         {
                          led[i].r=j;led[i].g=0;led[i].b=0;
                         ws2812_setleds(led,16);
                         _delay_us(50);
                         }
                        }
        }
    }
}
```

En este caso, lo que se hace es recorrer un *for* para poder ir manejando uno por uno los LED. Dentro de ese *for*, se anida otro, con el fin de generar los retardos en el encendido y el apagado de los led, variando la intensidad. Además se puede elegir el sentido horario y antihorario para el encendido de los led.

- ***Modulo Bluetooth***: Para usar dicho módulo, la conexión debe ir en un puerto que tenga conexión UART. En este caso el PORTE 0 y PORTE 1, corresponden al puerto RX y TX respectivamente.

  El pin TX del módulo debe ir al PORTE 0 (RX) y el pin RX del módulo debe ir al PORTE 1 (TX) de la placa, para que pueda enviar y recibir los datos.

  Para poder usar la transmisión por medio de la UART, usamos la librería *uart0.c y uart0.h*

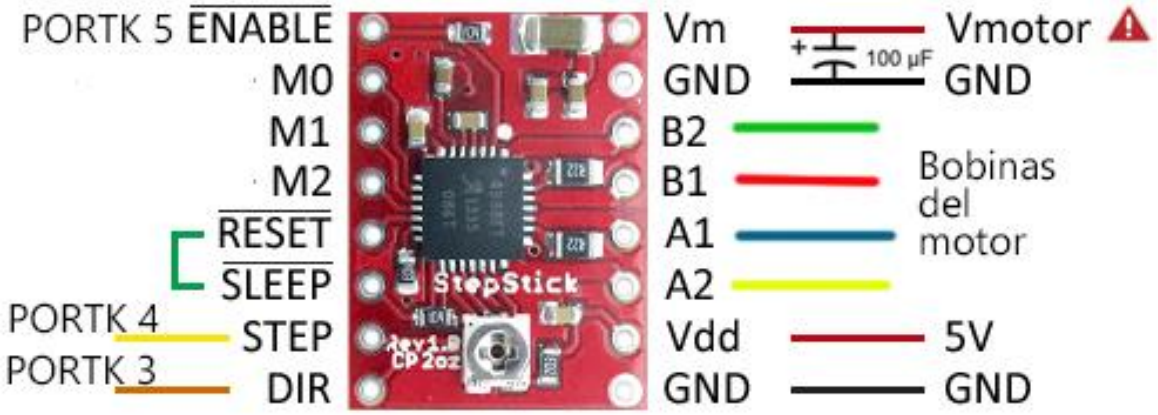
  Para establecer la conexión, los baudios deben estar sincronizados. Generalmente la configuración de baudios en las App de android de Bluetooth están definidas en 9600. Dado este valor, en el código se debe establecer el mismo valor de Baudios para que pueda sincronizarse la conexión y enviar y recibir datos sin inconvenientes. Para esto se debe modificar el registro UBRR0L, asignándole el número 51 para un micro que trabaja a 8MHz.

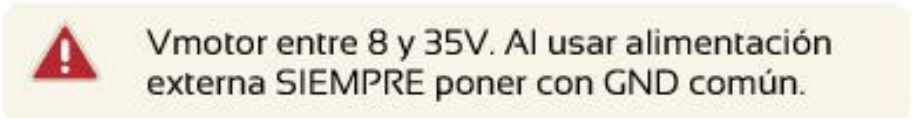
```
UBRR0L=51; // 9600 bps
```

- ***Módulo A4988:*** El módulo soporta hasta 2A de corriente, considerando que usamos un motor paso a paso bipolar que consume 1,8A estaría cubierto la tolerancia para poder usar dicho motor.

  Antes de usarlo se debe configurar y limitar el paso de la corriente para que funcione sin riesgo de generar daños en el motor y en el micro, modificando el potenciómetro que se encuentra entra los pines GND y DIR.

  Conexiones del módulo:





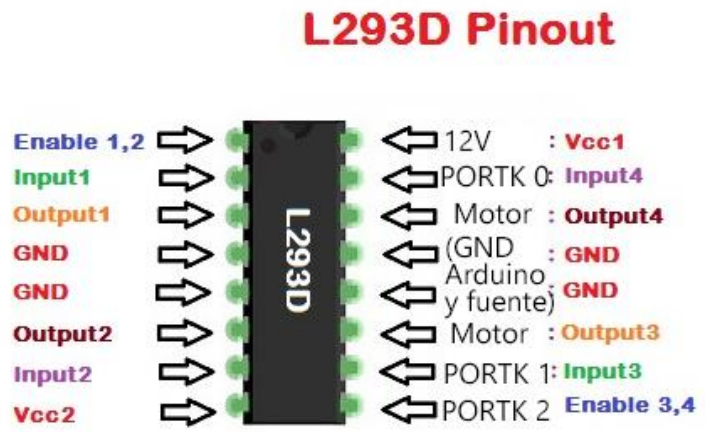
  Los *pines M0, M1 y M2* permiten definir el tipo de paso a usar. Nosotros lo usamos en paso completo, por lo que tanto M0, M1 y M2 están conectados a masa.
  --tablita pasos del motor--
  Para poder mover el motor, se debe habilitar con lógica negativa el ENABLE y deshabilitarlo cuando no se usa para no consumir y gastar recursos innecesarios.

  Con el *pin DIR* lo que se establece es la dirección en un sentido u otro, dependiendo las conexiones de las bobinas.

El *pin STEP* es el generador de pulsos que permiten el movimiento en el motor. En este caso, usamos un *for* con la cantidad de pasos que deseamos mover, en el que se alterna el envío de un pulso positivo y uno negativo para poder generar el movimiento.
Usando un delay en microsegundos lo que hacemos es variar o simular la velocidad con la que se mueve el motor.

```
for(int j=0;j<=CantidadPasos;j++){
                PAP_PASOS_ON;
                _delay_us(VELOCIDAD);
                PAP_PASOS_OFF;
                _delay_us(VELOCIDAD);
        }
```

- ***Actuador lineal 50mm:*** Para la utilización del actuador fue necesario usar un integrado L293D con puente H, que nos permite poder cambiar el sentido del motor, así lograr subir y bajar el actuador.



L293D Animation



L293D IC

Habilitando el *pin ENABLE* lo que hacemos es habilitar el funcionamiento del motor. Según el PORTK 0 y PORTK 1 lo que hacemos es habilitar un sentido u otro, permitiendo que el actuador suba o baje.

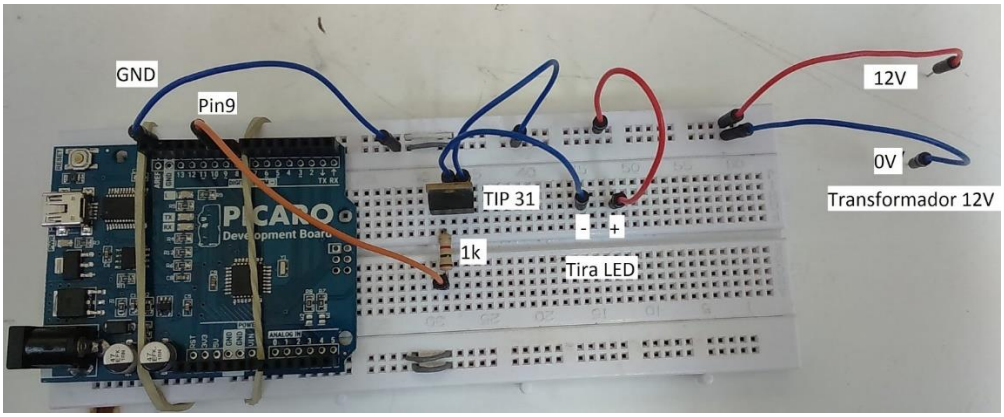
Para que el actuador suba se debe enviar un 0 en PORTK 0 y un 1 en el PORTK 1

```
PINK&=~(1<<0);PINK|=(1<<1);
```

Para que el actuador baje se debe enviar un 1 en PORTK 0 y un 0 en el PORTK 1

```
PINK&=~(1<<1);PINK|=(1<<0);
```

- ***Tira de LED RGB:*** En este caso como el consumo de las tiras es de 12v, se debe usar un circuito extra para poder controlar las tiras sin inconvenientes.
  El esquema que usamos lo sacamos de internet, de una página que explicaba el uso del transistor TIP31.



Una vez implementado el circuito en una plaqueta, el manejo del encendido de cada color es independiente. Activando un 1 el puerto se enciende el color habilitado y con 0 se apaga.

Fotos de la máquina:

